Syntactic Analysis

Roland Leißa
(based on slides by S. Hack, R. Wilhelm, and M. Sagiv)

University of Göttingen

Syntactic Analysis: Topics

- Introduction
  - The task of syntax analysis
  - Automatic generation
  - Error handling
- Context free grammars, derivations, and parse trees
- Pushdown automata
- Top-down syntax analysis
- Bottom-up syntax analysis

Syntax Analysis (Parsing)

- Functionality

  Input Sequence of symbols (tokens)

  Output Abstract Syntax Tree (AST) $\subseteq$ Parse Tree

- Report syntax errors, e.g. unbalanced parentheses
- Create “pretty-printed” version of the program (sometimes)
- In some cases the tree need not be generated (one-pass compilers)

Handling Syntax Errors

- Report and locate the error (symptom)
- Diagnose the error
- Correct the error
- Recover from the error in order to discover more errors (without reporting errors caused by others)

Example

```
a = a * (b + c * d ;
```

Error Diagnosis Data

- Line number (may be far from the actual error)
- The current symbol
- The symbols expected in the current parser state

Example Context Free Grammar (Section)

Stat	→	If_Stat
	|	While_Stat
	|	Repeat_Stat
	|	Proc_Call
	|	Assignment
If_Stat	→	**if** Cond **then** Stat_Seq **else** Stat_Seq **fi**
	|	**if** Cond **then** Stat_Seq **fi**
While_Stat	→	**while** Cond **do** Stat_Seq **od**
Repeat_Stat	→	**repeat** Stat_Seq **until** Cond
Proc_Call	→	Name **(** Expr_Seq **)**
Assignment	→	Name := Expr
Stat_Seq	→	Stat
	|	Stat_Seq; Stat
Expr_Seq	→	Expr
	|	Expr_Seq, Expr

Context Free Grammars: Definition

A context-free-grammar is a quadruple $G = (V_N, V_T, \to, S)$ where:

- V_N: finite set of nonterminals (denoted by large Roman letters in the following)
- V_T: finite set of terminals
- finite relation of productions: literature: $P \subseteq V_N \times (V_N \cup V_T)^*$
  $N \to A_1 \cdots A_n$, where $N \in V_N$, $A_i \in V_N \cup V_T$, $n \geq 0$
- $S \in V_N$: the start nonterminal

Notation

- Greek letters stand for a sequence of terminals/nonterminals
- Empty right-hand side: $N \to \varepsilon$
-

$$\begin{array}{lcl} N & \to & \varphi_1 \\ & | & \varphi_2 \end{array} \quad := \quad \begin{array}{lcl} N & \to & \varphi_1 \\ N & \to & \varphi_2 \end{array}$$

Examples

$$G_0 = (\{E, T, F\}, \{+, *, (,), \mathsf{id}\}, \rightarrow, E)$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \mathsf{id}$$

$$G_1 = (\{E\}, \{+, *, (,), \mathsf{id}\}, \rightarrow, E)$$

$$E \rightarrow E + E \mid E * E \mid (E) \mid \mathsf{id}$$

Derivations

Given a context-free-grammar $G = (V_N, V_T, \rightarrow, S)$

$$\textsc{Step}\ \frac{A \rightarrow \alpha}{\varphi_1\, A\, \varphi_2 \implies \varphi_1\, \alpha\, \varphi_2}$$

The language defined by G

$$L(G) = \{w \in V_T^* \mid S \overset{*}{\implies} w\}$$

Reduced and Extended Context Free Grammars

A nonterminal A is

reachable: There exist φ_1, φ_2 such that $S \overset{*}{\Longrightarrow} \varphi_1 A \varphi_2$

productive: There exists $w \in V_T^*$, $A \overset{*}{\Longrightarrow} w$

Removing unreachable and non-productive nonterminals and the productions they occur in doesn't change the defined language

A grammar is reduced if it has neither unreachable nor non-productive nonterminals

A grammar is extended if a new startsymbol S' and a new production $S' \to S$ are added to the grammar

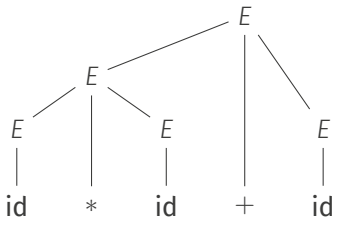
From now on, we only consider reduced and extended grammars

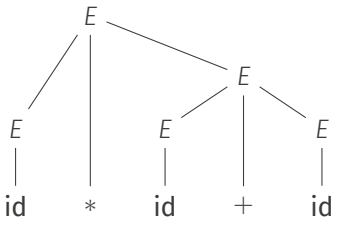
Parse Tree (Concrete Syntax Tree)

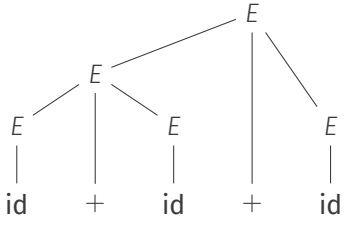
- Each derivation $S \overset{+}{\Longrightarrow} w$ gives rise to an (ordered) parse tree
- Root is labeled with S
- Internal nodes are labeled by nonterminals
- Leaves are labeled by terminals or by ε
- If $S \overset{+}{\Longrightarrow} w$ contains step $\alpha A \gamma \Longrightarrow \alpha \beta \gamma$
  then there is an inner node A with children β

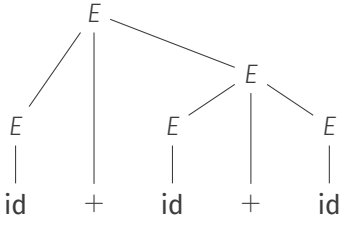
Examples

$E \to E + E \mid E * E \mid (E) \mid \mathsf{id}$









Leftmost/Rightmost Derivations

Given a context-free grammar $G = (V_N, V_T, \rightarrow, S)$

$$\textsc{LeftMost}\ \frac{A \rightarrow \alpha \qquad \varphi_1 \in V_T^* \qquad \varphi_2 \in (V_N \cup V_T)^*}{\varphi_1 A \varphi_2 \underset{lm}{\Longrightarrow} \varphi_1 \alpha \varphi_2}$$

$$\textsc{RightMost}\ \frac{A \rightarrow \alpha \qquad \varphi_2 \in V_T^* \qquad \varphi_1 \in (V_N \cup V_T)^*}{\varphi_1 A \varphi_2 \underset{rm}{\Longrightarrow} \varphi_1 \alpha \varphi_2}$$

Ambiguous Grammars

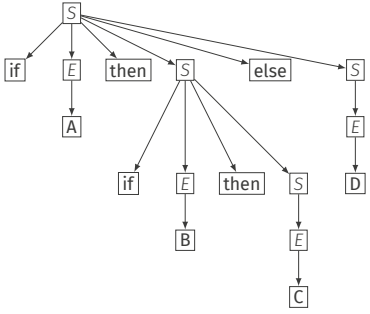
- A grammar that has (equivalently)
  - two leftmost derivations for the same string,
  - two rightmost derivations for the same string,
  - two syntax trees for the same string.

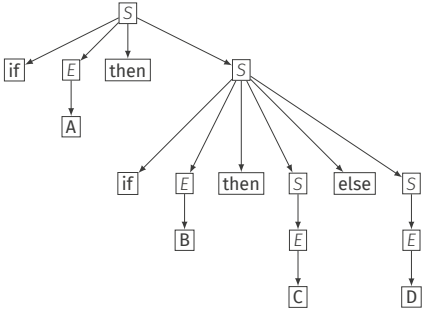
  is called ambiguous.
- It is undecidable if a grammar is ambiguous or not
- There are unambiguous grammars (whose languages) cannot be accepted with a deterministic push-down automaton
- For parsing, we're interested in grammars that can be accepted with a deterministic push-down automaton

Ambiguous Grammars: Dangling Else

$$
\begin{aligned}
S &\rightarrow \textbf{if } E \textbf{ then } S \\
&\mid \textbf{if } E \textbf{ then } S \textbf{ else } S \\
&\mid E \\
E &\rightarrow \mathsf{A} \mid \mathsf{B} \mid \mathsf{C} \mid \mathsf{D}
\end{aligned}
$$

if A then if B then C else D has 2 different syntax trees





In practice the **else** always belongs to the last **if**

Ambiguous Grammars: Dangling-Else

Possible Solutions

- Change language: add a closing **end**
- Change grammar by factoring:

$$
\begin{aligned}
S &\to \textbf{if } E \textbf{ then } S \\
&\mid \textbf{if } E \textbf{ then } S' \textbf{ else } S \\
&\mid E \\
S' &\to \textbf{if } E \textbf{ then } S' \textbf{ else } S' \\
&\mid E \\
E &\to \mathsf{A} \mid \mathsf{B} \mid \mathsf{C} \mid \mathsf{D}
\end{aligned}
$$

  Con: cumbersome and messy; see Java Spec:
  `https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.5`

- Leave grammar ambiguous and make sure the parser does “the right thing”

Ambiguous Grammars: An Example From C++

Delcaration or Expression?

```
class F {
  int func() {
    // decl. foo<bar> f;
    foo<bar>f;
  }
  class bar;
  template<typename T>
  class foo {};
};
```

```
class F {
  int func() {
    // expr. (foo<bar) > f;
    foo<bar>f;
  }
  int foo, bar;
  bool f;
};
```

- Syntactically, declarations and expressions are not disjoint
- Context that helps to discriminate them might only come later
- How to implement a parser for that?
  - First parse method signatures and classes and collect declarations
  - Then, parse method bodies

C++: Parsing Undecidable

```
TuringMachine<42>::T * B;
```

Curiosity: Syntax error if constant is no prime number

http://stackoverflow.com/questions/14589346/is-c-context-free-or-context-sensitive